Infosys®
Navigate your next

# COURSE COMPLETION CERTIFICATE

The certificate is awarded to

## syed kareem

for successfully completing the course

**Digital Electronics- Online Digital Electronics Courses**

on November 4, 2025

Infosys | Springboard

*Congratulations! You make us proud!*

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

Issued on: Wednesday, November 5, 2025
To verify, scan the QR code at https://verify.onwingspan.com

# PRAGYAN AI PYTHON NOTES

## 1. Variables and Data Types

**Step 1: Definition**Variables are reserved memory locations used to store different types of values (like numbers, text, or objects) 3, 4. Python is dynamically typed, meaning you do not explicitly declare the type of a variable before using it 5, 6. Core data types include Integer (whole numbers), Float (decimal numbers), Boolean (True/False), and Strings (text) 7-9.
**Step 2: Why we are using that**Variables are the foundational building blocks of programming, allowing us to store, retrieve, manipulate, and reuse data dynamically throughout a program's lifecycle 3, 4.
**Step 3: How to use that**You create a variable by assigning a value to a name using the assignment operator (=) 6, 10. Variable names must start with a letter or underscore and are case-sensitive 10.
**Step 4: Give one practical example on that**

```
age = 25                        # Integer data type
price = 50.5                    # Float data type
is_active = True                # Boolean data type
message = "Welcome to Python" # String data type
```

## 2. Input and Output Statements

**Step 1: Definition**The input() function prompts the user to enter data via the console and returns it as a string, while the print() function displays output back to the console 11, 12.
**Step 2: Why we are using that**They are essential for building interactive applications where the system must communicate with the user to request information and present calculated results 11, 13.
**Step 3: How to use that**Call input("Prompt") to pause execution and read user input 11. To output data, call print(data). You can format strings elegantly using f-strings (prefixing the string with f and enclosing variables in {}) 12.
**Step 4: Give one practical example on that**

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # Typecasting string to integer
print(f"My name is {name} and I am {age} years old.")
```

## 3. Python Operators

Operators perform operations on values and variables 14, 15. Python contains several types of operators, detailed below.

PRAGYAN AI

### 3.1 Arithmetic Operators

**Step 1: Definition**Arithmetic operators are used to perform mathematical operations: Addition (+), Subtraction (-), Multiplication (*), Division (/), Floor Division (//), Modulo/Remainder (%), and Exponentiation/Power (**) 14, 16.
**Step 2: Why we are using that**To perform basic to advanced mathematical calculations dynamically within our code 16.
**Step 3: How to use that**Place the operator symbol between two numeric operands 16, 17.
**Step 4: Give one practical example on that**

```
a, b = 10, 3
print(a + b)  # 13 (Addition)
print(a / b)  # 3.333 (Division)
print(a // b) # 3 (Floor Division - rounds down)
print(a % b)  # 1 (Modulo - remainder)
print(a ** b) # 1000 (10 to the power of 3)
```

### 3.2 Assignment Operators

**Step 1: Definition**Assignment operators are used to assign values to variables: =, +=, -=, *=, /=, %=, **= 18.
**Step 2: Why we are using that**To initialize or update the value of a variable in a concise manner (e.g., incrementing a counter) 18.
**Step 3: How to use that**Place the variable on the left and the value/expression on the right (e.g., a += 1 is shorthand for a = a + 1) 18.
**Step 4: Give one practical example on that**

```
x = 5
x += 3 # x becomes 8
x *= 2 # x becomes 16
```

### 3.3 Comparison (Relational) Operators

**Step 1: Definition**Comparison operators compare two values and return a boolean result (True or False): Equal (==), Not Equal (!=), Greater Than (>), Less Than (<), Greater/Equal (>=), Less/Equal (<=) 19, 20.
**Step 2: Why we are using that**They evaluate conditions to make decisions in loops and conditional statements 20.
**Step 3: How to use that**Place the operator between two operands. The interpreter evaluates the relationship and returns a boolean 20.
**Step 4: Give one practical example on that**

```
a, b = 5, 10
print(a == b) # False
print(a != b) # True
print(a < b)  # True
```

PRAGYAN AI

### 3.4 Logical Operators

**Step 1: Definition**Logical operators combine conditional statements: and (True if both are true), or (True if at least one is true), and not (reverses the boolean state) 19, 21.
**Step 2: Why we are using that**To check multiple conditions simultaneously for complex decision-making 21.
**Step 3: How to use that**Place and/or between two boolean expressions, or place not before a single expression 21.
**Step 4: Give one practical example on that**

```
x = 5
print(x > 3 and x < 10)  # True (Both conditions met)
print(x > 3 or x > 10)   # True (One condition met)
print(not(x > 3))        # False (Reverses True to False)
```

### 3.5 Bitwise Operators

**Step 1: Definition**Bitwise operators act on operands bit-by-bit: AND (&), OR (|), NOT (~), XOR (^), Left Shift (<<), Right Shift (>>) 19, 22.
**Step 2: Why we are using that**To perform low-level operations directly on binary numbers 19.
**Step 3: How to use that**Apply them to integers; Python evaluates them at the binary (bit) level 19, 22.
**Step 4: Give one practical example on that**

```
a = 10 # Binary: 1010
b = 4  # Binary: 0100
print(a & b) # 0 (Binary AND)
print(a | b) # 14 (Binary OR)
```

### 3.6 Identity Operators

**Step 1: Definition**Identity operators (is, is not) check if two values are located on the exact same part of the computer's memory 22, 23.
**Step 2: Why we are using that**To verify if two variables reference the exact same object, not just if their values are equal 22, 23.
**Step 3: How to use that**Use x is y or x is not y to evaluate memory identity 23.
**Step 4: Give one practical example on that**

```
x1, y1 = 5, 5
print(x1 is y1) # True (Integers map to same memory)
list1, list2 = [24, 25]
print(list1 is list2) # False (Equal values, but separate objects in
memory)
```

### 3.7 Membership Operators

**Step 1: Definition**Membership operators (in, not in) test whether a value or variable is found inside a sequence (like a string, list, tuple, or dictionary) 22, 26.
**Step 2: Why we are using that**To quickly validate if a specific item exists within a collection without writing manual search loops 26.

PRAGYAN AI

**Step 3: How to use that**Use value in sequence to return True if it exists, or False otherwise 27.
**Step 4: Give one practical example on that**

```
message = "Hello World"
print('H' in message) # True
print('z' not in message) # True
```

## 4. Control Flow (Selective Statements)

**Step 1: Definition**Selective statements control the flow of a program based on evaluating boolean conditions. They include if, if-else, if-elif-else, and nested if statements 28, 29.
**Step 2: Why we are using that**To enable decision-making in our software, allowing the program to execute different logic based on varying inputs or states 28, 29.
**Step 3: How to use that**Use if condition: followed by an indented block. Provide fallback logic using elif (else if) for multiple conditions, and an else block for when all conditions fail 30-32.
**Step 4: Give one practical example on that**

```
age = int(input("Enter Age: "))
if age < 18:
    print("Minor")
elif age >= 18 and age < 65:
    print("Adult")
else:
    print("Senior")
```

## 5. Iterative & Jump Statements (Loops)

**Step 1: Definition**Iterative statements repeat a block of code. The for loop iterates over a sequence (lists, strings, ranges), while the while loop reiterates as long as a condition is true 33-37. Jump statements (break, continue, pass) alter loop execution 38, 39.
**Step 2: Why we are using that****Loops to automate** repetitive tasks. Jump statements provide fine-grained control: break completely terminates a loop, continue skips the current iteration, and pass acts as an empty placeholder 38-42.
**Step 3: How to use that**Use for item in sequence: or while condition:. Inside the loop, insert break or continue coupled with if conditions to manipulate the flow 35, 38, 39.
**Step 4: Give one practical example on that**

```
# For loop with Continue
for i in range(1, 6):
    if i == 3:
        continue # Skips printing 3
    print(i) # Prints 1, 2, 4, 5

# While loop with Break
count = 1
while True:
    print(count)
    if count == 3:
```

PRAGYAN AI

```
        break # Terminates loop at 3
    count += 1
```

## 6. String Handling & Methods

**Step 1: Definition**A string is an immutable sequence of Unicode characters 40, 43. Python provides built-in methods to manipulate strings: isalnum(), isalpha(), isdigit(), islower(), isspace(), istitle(), isupper(), replace(), split(), count(), and find() 44, 45.
**Step 2: Why we are using that**To format, validate, clean, and extract insights from textual data easily without building custom logic 44.
**Step 3: How to use that**Access characters via slicing string[start:stop:step] 46, 47. Call methods using dot notation: string.method_name() 45, 48
**Step 4: Give one practical example on that**

```
text = "Python Programming 101"
print(text[0:6])                # Slicing: 'Python'
print(text.isupper())           # False
print(text.replace("101", "Masterclass")) # 'Python Programming
Masterclass'
print(text.split())             # ['Python', 'Programming', '101']
print(text.count('o'))          # 2
```

## 7. Built-in Data Structures

### 7.1 Lists and List Methods

**Step 1: Definition**A list is an ordered, mutable collection that allows duplicate elements 49, 50. Comprehensive methods include: append(), clear(), copy(), count(), extend(), index(), insert(), pop(), remove(), reverse(), and sort() 51, 52.
**Step 2: Why we are using that**To dynamically store and manipulate an ordered sequence of varying data types (e.g., adding or removing items on the fly) 49, 53.
**Step 3: How to use that**Declare lists using []. Call methods via list_name.method() 49, 51. You can also use List Comprehension [expr for item in iterable] for concise creation 54.
**Step 4: Give one practical example on that**

```
data = [25, 55, 56]
data.append(10)         # Adds 10: [25, 55-57]
data.insert(1, 15)      # Inserts at index 1: [2, 25, 55-57]
data.extend([24, 58])    # Appends sequence: [2, 24, 25, 55-58]
data.remove(2)          # Removes value 2
popped = data.pop()     # Removes and returns last item (3)
data.sort()             # Sorts ascending: [2, 24, 55-57]
data.reverse()          # Reverses order: [2, 24, 55-57]
print(data.count(10)) # Returns 1
```

PRAGYAN AI

### 7.2 Tuples and Tuple Methods

**Step 1: Definition**A tuple is an ordered, *immutable* (unchangeable) collection defined using parentheses () 59-61. Key methods are count() and index(), and it supports len() 62, 63.
**Step 2: Why we are using that**To safeguard data that must not change post-creation (e.g., geographic coordinates). They are faster and more memory-efficient than lists 60, 61.
**Step 3: How to use that**Create via (). Read values via indexing. Because they are immutable, packing (grouping items) and unpacking (assigning items to variables) are common operations 64, 65.
**Step 4: Give one practical example on that**

```
dimensions = (1920, 1080, 1080)
print(dimensions.count(1080)) # Returns 2
print(dimensions.index(1920)) # Returns index 0
# Unpacking
w, h, depth = dimensions
```

### 7.3 Sets and Set Methods

**Step 1: Definition**A set is an unordered collection of *unique* elements 66, 67. Operations include add(), remove(), discard(), Union (|), Intersection (&), Difference (-), and Symmetric Difference (^) 66, 68, 69.
**Step 2: Why we are using that**To perform mathematical set operations rapidly and eliminate duplicate values from other collections 68.
**Step 3: How to use that**Define using {} or set(). Apply mathematical operations using their respective methods or operator symbols 66, 68.
**Step 4: Give one practical example on that**

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set1.add(6)             # {1, 2, 3, 6}
set1.discard(6)         # Removes 6 safely
print(set1 | set2)      # Union: {1, 2, 3, 4, 5}
print(set1 & set2)      # Intersection: {3}
print(set1 - set2)      # Difference: {1, 2}
```

### 7.4 Dictionaries and Dictionary Methods

**Step 1: Definition**A dictionary is an unordered, mutable collection of key-value pairs 70, 71. Methods include: clear(), copy(), fromkeys(), get(), items(), keys(), pop(), popitem(), setdefault(), update(), and values() 72-75.
**Step 2: Why we are using that**To associate mapping data (like an ID to a Name) ensuring highly optimized data retrieval via unique keys rather than numeric indexes 70, 71.
**Step 3: How to use that**Define via {key: value}. Use methods to extract keys, values, or safely get items without causing errors 74, 76.
**Step 4: Give one practical example on that**

```
user = {"name": "Alice", "role": "Admin"}
```

PRAGYAN AI

```
print(user.get("age", "Not Found")) # Safely gets 'age' or defaults
to 'Not Found'
user.update({"age": 30, "city": "NYC"}) # Adds/updates multiple pairs
print(user.keys())   # dict_keys(['name', 'role', 'age', 'city'])
print(user.values()) # dict_values(['Alice', 'Admin', 30, 'NYC'])
print(user.items())  # Returns all key-value tuples
user.pop("city")     # Removes key 'city'
```

## 8. Functions and Arguments

**Step 1: Definition**A function is a modular, reusable block of code defined with the def keyword 77-79. Functions can take inputs (Arguments: Positional, Keyword, Default, and Arbitrary *args/**kwargs) and optionally return values 80-85. A function calling itself is known as Recursion 86, 87.
**Step 2: Why we are using that**To promote code modularity, increase readability, and prevent repetition (Code Reusability). Recursion simplifies complex problems like math factorials 78, 86.
**Step 3: How to use that**Use def func_name(parameters): followed by logic and an optional return statement. Supply default values (arg=value) for Default arguments, pass explicit names (name="Bob") for Keyword arguments, or use *args to capture an unlimited list of arguments 82, 84, 88.
**Step 4: Give one practical example on that**

```
# Function with Default, Arbitrary args, and Return
def process_data(multiplier=2, *numbers):
    result = []
    for num in numbers:
        result.append(num * multiplier)
    return result

# Calling with Positional and Arbitrary arguments
output = process_data(3, 10, 20, 30)
print(output) # [14, 76, 83]

# Recursive Function
def factorial(n):
    if n == 1 or n == 0: return 1
    return n * factorial(n - 1)
```

## 9. Object-Oriented Programming (OOP) Concepts

**Step 1: Definition**OOP is a paradigm designing software around "Objects" (data entities with state) and "Classes" (blueprints) 89, 90. The four core pillars are:

1. **Encapsulation:** Wrapping variables and methods into a single unit (hiding data) 91, 92.
2. **Inheritance:** A child class acquiring properties of a parent class (Single, Multiple, Multilevel, Hierarchical, Hybrid) 93-95.

PRAGYAN AI

3. **Polymorphism:** Using a single entity/method to represent different behaviors depending on context (Method Overriding and Overloading) 93, 96, 97.
4. **Abstraction:** Hiding complex internal details and showing only essential features 91.**Step 2: Why we are using that**To build large, scalable, and maintainable programs. It enhances code security (Encapsulation), aggressive code reuse (Inheritance), and flexibility (Polymorphism) 98, 99.**Step 3: How to use that**Define a class. Use the __init__ constructor to initialize state. Pass self to instance methods. Use super() to trigger parent logic in inheritance 100, 101.**Step 4: Give one practical example on that**

```
# Encapsulation & Class definition
class Vehicle:
    def __init__(self, name):
        self.name = name # Public attribute
        self.__engine_status = "Off" # Encapsulated (Private)
attribute

    def start_engine(self):
        self.__engine_status = "On"
        return f"{self.name} engine is {self.__engine_status}"

# Inheritance & Polymorphism (Overriding)
class Car(Vehicle):
    def __init__(self, name, wheels):
        super().__init__(name) # Inheriting parent constructor
        self.wheels = wheels

    # Polymorphism: Overriding parent behavior if it existed
    def start_engine(self):
        return super().start_engine() + f" with {self.wheels} wheels
ready!"

my_car = Car("Mustang", 4)
print(my_car.start_engine())
```

## 10. Errors and Exception Handling

**Step 1: Definition**Syntax errors occur due to bad formatting (parsing errors), whereas Exceptions (like ZeroDivisionError, ValueError) are runtime disruptions 58, 97, 102, 103. Handling involves intercepting these exceptions using try, except, and finally 95, 104.**Step 2: Why we are using that**If an exception is unhandled, a program will crash abruptly. Exception handling safeguards system resources, ensures graceful failure, and keeps applications running 95, 104, 105.**Step 3: How to use that**Wrap risky code inside try:. Catch specific or general errors using except ExceptionType as e:. Use finally: for cleanup code (e.g., closing files) that strictly executes regardless of whether an error happened 104, 106, 107.**Step 4: Give one practical example on that**

```
try:
```

PRAGYAN AI

```
    x = int(input("Enter number: "))
    result = 10 / x
    print(result)
except ZeroDivisionError as e:
    print("Cannot divide by zero!", e)
except ValueError:
    print("Please enter a valid integer!")
finally:
    print("Execution complete. Cleaning up resources.")
```

## 11. Python Decorators

**Step 1: Definition**Decorators are higher-order functions that modify or extend the behavior of another function without explicitly changing its code 108. They wrap the target function using inner functions 108.**Step 2: Why we are using that**To dynamically add syntactic sugar and reusable functionality (like logging, timing execution, authentication) cleanly and transparently across multiple functions 109, 110.**Step 3: How to use that**Define a function that takes a function as an argument, defines a wrapper function inside it, and returns the wrapper 108. Apply it instantly using the pie syntax @decorator_name directly above the target function 111. Use @functools.wraps to preserve the original function's metadata 112.**Step 4: Give one practical example on that**

```
import functools

# The Decorator
def my_logger(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Executing {func.__name__}...")
        result = func(*args, **kwargs)
        print("Execution finished.")
        return result
    return wrapper

# Applying the decorator
@my_logger
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Architect")
# Output:
# Executing say_hello...
# Hello, Architect!
# Execution finished.
```

PRAGYAN AI

Infosys®
Navigate your next

# COURSE COMPLETION CERTIFICATE

The certificate is awarded to

## syed kareem

for successfully completing the course

**Digital Electronics- Online Digital Electronics Courses**

on November 4, 2025

Infosys | Springboard

*Congratulations! You make us proud!*

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

Issued on: Wednesday, November 5, 2025
To verify, scan the QR code at https://verify.onwingspan.com

# PRAGYAN AI PYTHON NOTES

## 1. Variables and Data Types

**Step 1: Definition**Variables are reserved memory locations used to store different types of values (like numbers, text, or objects) 3, 4. Python is dynamically typed, meaning you do not explicitly declare the type of a variable before using it 5, 6. Core data types include Integer (whole numbers), Float (decimal numbers), Boolean (True/False), and Strings (text) 7-9.
**Step 2: Why we are using that**Variables are the foundational building blocks of programming, allowing us to store, retrieve, manipulate, and reuse data dynamically throughout a program's lifecycle 3, 4.
**Step 3: How to use that**You create a variable by assigning a value to a name using the assignment operator (=) 6, 10. Variable names must start with a letter or underscore and are case-sensitive 10.
**Step 4: Give one practical example on that**

```
age = 25                        # Integer data type
price = 50.5                    # Float data type
is_active = True                # Boolean data type
message = "Welcome to Python" # String data type
```

## 2. Input and Output Statements

**Step 1: Definition**The input() function prompts the user to enter data via the console and returns it as a string, while the print() function displays output back to the console 11, 12.
**Step 2: Why we are using that**They are essential for building interactive applications where the system must communicate with the user to request information and present calculated results 11, 13.
**Step 3: How to use that**Call input("Prompt") to pause execution and read user input 11. To output data, call print(data). You can format strings elegantly using f-strings (prefixing the string with f and enclosing variables in {}) 12.
**Step 4: Give one practical example on that**

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # Typecasting string to integer
print(f"My name is {name} and I am {age} years old.")
```

## 3. Python Operators

Operators perform operations on values and variables 14, 15. Python contains several types of operators, detailed below.

PRAGYAN AI

### 3.1 Arithmetic Operators

**Step 1: Definition**Arithmetic operators are used to perform mathematical operations: Addition (+), Subtraction (-), Multiplication (*), Division (/), Floor Division (//), Modulo/Remainder (%), and Exponentiation/Power (**) 14, 16.
**Step 2: Why we are using that**To perform basic to advanced mathematical calculations dynamically within our code 16.
**Step 3: How to use that**Place the operator symbol between two numeric operands 16, 17.
**Step 4: Give one practical example on that**

```
a, b = 10, 3
print(a + b)  # 13 (Addition)
print(a / b)  # 3.333 (Division)
print(a // b) # 3 (Floor Division - rounds down)
print(a % b)  # 1 (Modulo - remainder)
print(a ** b) # 1000 (10 to the power of 3)
```

### 3.2 Assignment Operators

**Step 1: Definition**Assignment operators are used to assign values to variables: =, +=, -=, *=, /=, %=, **= 18.
**Step 2: Why we are using that**To initialize or update the value of a variable in a concise manner (e.g., incrementing a counter) 18.
**Step 3: How to use that**Place the variable on the left and the value/expression on the right (e.g., a += 1 is shorthand for a = a + 1) 18.
**Step 4: Give one practical example on that**

```
x = 5
x += 3 # x becomes 8
x *= 2 # x becomes 16
```

### 3.3 Comparison (Relational) Operators

**Step 1: Definition**Comparison operators compare two values and return a boolean result (True or False): Equal (==), Not Equal (!=), Greater Than (>), Less Than (<), Greater/Equal (>=), Less/Equal (<=) 19, 20.
**Step 2: Why we are using that**They evaluate conditions to make decisions in loops and conditional statements 20.
**Step 3: How to use that**Place the operator between two operands. The interpreter evaluates the relationship and returns a boolean 20.
**Step 4: Give one practical example on that**

```
a, b = 5, 10
print(a == b) # False
print(a != b) # True
print(a < b)  # True
```

PRAGYAN AI

### 3.4 Logical Operators

**Step 1: Definition**Logical operators combine conditional statements: and (True if both are true), or (True if at least one is true), and not (reverses the boolean state) 19, 21.
**Step 2: Why we are using that**To check multiple conditions simultaneously for complex decision-making 21.
**Step 3: How to use that**Place and/or between two boolean expressions, or place not before a single expression 21.
**Step 4: Give one practical example on that**

```
x = 5
print(x > 3 and x < 10)  # True (Both conditions met)
print(x > 3 or x > 10)   # True (One condition met)
print(not(x > 3))        # False (Reverses True to False)
```

### 3.5 Bitwise Operators

**Step 1: Definition**Bitwise operators act on operands bit-by-bit: AND (&), OR (|), NOT (~), XOR (^), Left Shift (<<), Right Shift (>>) 19, 22.
**Step 2: Why we are using that**To perform low-level operations directly on binary numbers 19.
**Step 3: How to use that**Apply them to integers; Python evaluates them at the binary (bit) level 19, 22.
**Step 4: Give one practical example on that**

```
a = 10 # Binary: 1010
b = 4  # Binary: 0100
print(a & b) # 0 (Binary AND)
print(a | b) # 14 (Binary OR)
```

### 3.6 Identity Operators

**Step 1: Definition**Identity operators (is, is not) check if two values are located on the exact same part of the computer's memory 22, 23.
**Step 2: Why we are using that**To verify if two variables reference the exact same object, not just if their values are equal 22, 23.
**Step 3: How to use that**Use x is y or x is not y to evaluate memory identity 23.
**Step 4: Give one practical example on that**

```
x1, y1 = 5, 5
print(x1 is y1) # True (Integers map to same memory)
list1, list2 = [24, 25]
print(list1 is list2) # False (Equal values, but separate objects in
memory)
```

### 3.7 Membership Operators

**Step 1: Definition**Membership operators (in, not in) test whether a value or variable is found inside a sequence (like a string, list, tuple, or dictionary) 22, 26.
**Step 2: Why we are using that**To quickly validate if a specific item exists within a collection without writing manual search loops 26.

PRAGYAN AI

**Step 3: How to use that**Use value in sequence to return True if it exists, or False otherwise 27.
**Step 4: Give one practical example on that**

```
message = "Hello World"
print('H' in message) # True
print('z' not in message) # True
```

## 4. Control Flow (Selective Statements)

**Step 1: Definition**Selective statements control the flow of a program based on evaluating boolean conditions. They include if, if-else, if-elif-else, and nested if statements 28, 29.
**Step 2: Why we are using that**To enable decision-making in our software, allowing the program to execute different logic based on varying inputs or states 28, 29.
**Step 3: How to use that**Use if condition: followed by an indented block. Provide fallback logic using elif (else if) for multiple conditions, and an else block for when all conditions fail 30-32.
**Step 4: Give one practical example on that**

```
age = int(input("Enter Age: "))
if age < 18:
    print("Minor")
elif age >= 18 and age < 65:
    print("Adult")
else:
    print("Senior")
```

## 5. Iterative & Jump Statements (Loops)

**Step 1: Definition**Iterative statements repeat a block of code. The for loop iterates over a sequence (lists, strings, ranges), while the while loop reiterates as long as a condition is true 33-37. Jump statements (break, continue, pass) alter loop execution 38, 39.
**Step 2: Why we are using that****Loops to automate** repetitive tasks. Jump statements provide fine-grained control: break completely terminates a loop, continue skips the current iteration, and pass acts as an empty placeholder 38-42.
**Step 3: How to use that**Use for item in sequence: or while condition:. Inside the loop, insert break or continue coupled with if conditions to manipulate the flow 35, 38, 39.
**Step 4: Give one practical example on that**

```
# For loop with Continue
for i in range(1, 6):
    if i == 3:
        continue # Skips printing 3
    print(i) # Prints 1, 2, 4, 5

# While loop with Break
count = 1
while True:
    print(count)
    if count == 3:
```

PRAGYAN AI

```
        break # Terminates loop at 3
    count += 1
```

## 6. String Handling & Methods

**Step 1: Definition**A string is an immutable sequence of Unicode characters 40, 43. Python provides built-in methods to manipulate strings: isalnum(), isalpha(), isdigit(), islower(), isspace(), istitle(), isupper(), replace(), split(), count(), and find() 44, 45.
**Step 2: Why we are using that**To format, validate, clean, and extract insights from textual data easily without building custom logic 44.
**Step 3: How to use that**Access characters via slicing string[start:stop:step] 46, 47. Call methods using dot notation: string.method_name() 45, 48
**Step 4: Give one practical example on that**

```
text = "Python Programming 101"
print(text[0:6])                # Slicing: 'Python'
print(text.isupper())           # False
print(text.replace("101", "Masterclass")) # 'Python Programming
Masterclass'
print(text.split())             # ['Python', 'Programming', '101']
print(text.count('o'))          # 2
```

## 7. Built-in Data Structures

### 7.1 Lists and List Methods

**Step 1: Definition**A list is an ordered, mutable collection that allows duplicate elements 49, 50. Comprehensive methods include: append(), clear(), copy(), count(), extend(), index(), insert(), pop(), remove(), reverse(), and sort() 51, 52.
**Step 2: Why we are using that**To dynamically store and manipulate an ordered sequence of varying data types (e.g., adding or removing items on the fly) 49, 53.
**Step 3: How to use that**Declare lists using []. Call methods via list_name.method() 49, 51. You can also use List Comprehension [expr for item in iterable] for concise creation 54.
**Step 4: Give one practical example on that**

```
data = [25, 55, 56]
data.append(10)         # Adds 10: [25, 55-57]
data.insert(1, 15)      # Inserts at index 1: [2, 25, 55-57]
data.extend([24, 58])    # Appends sequence: [2, 24, 25, 55-58]
data.remove(2)          # Removes value 2
popped = data.pop()     # Removes and returns last item (3)
data.sort()             # Sorts ascending: [2, 24, 55-57]
data.reverse()          # Reverses order: [2, 24, 55-57]
print(data.count(10)) # Returns 1
```

PRAGYAN AI

### 7.2 Tuples and Tuple Methods

**Step 1: Definition**A tuple is an ordered, *immutable* (unchangeable) collection defined using parentheses () 59-61. Key methods are count() and index(), and it supports len() 62, 63.
**Step 2: Why we are using that**To safeguard data that must not change post-creation (e.g., geographic coordinates). They are faster and more memory-efficient than lists 60, 61.
**Step 3: How to use that**Create via (). Read values via indexing. Because they are immutable, packing (grouping items) and unpacking (assigning items to variables) are common operations 64, 65.
**Step 4: Give one practical example on that**

```
dimensions = (1920, 1080, 1080)
print(dimensions.count(1080)) # Returns 2
print(dimensions.index(1920)) # Returns index 0
# Unpacking
w, h, depth = dimensions
```

### 7.3 Sets and Set Methods

**Step 1: Definition**A set is an unordered collection of *unique* elements 66, 67. Operations include add(), remove(), discard(), Union (|), Intersection (&), Difference (-), and Symmetric Difference (^) 66, 68, 69.
**Step 2: Why we are using that**To perform mathematical set operations rapidly and eliminate duplicate values from other collections 68.
**Step 3: How to use that**Define using {} or set(). Apply mathematical operations using their respective methods or operator symbols 66, 68.
**Step 4: Give one practical example on that**

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set1.add(6)             # {1, 2, 3, 6}
set1.discard(6)         # Removes 6 safely
print(set1 | set2)      # Union: {1, 2, 3, 4, 5}
print(set1 & set2)      # Intersection: {3}
print(set1 - set2)      # Difference: {1, 2}
```

### 7.4 Dictionaries and Dictionary Methods

**Step 1: Definition**A dictionary is an unordered, mutable collection of key-value pairs 70, 71. Methods include: clear(), copy(), fromkeys(), get(), items(), keys(), pop(), popitem(), setdefault(), update(), and values() 72-75.
**Step 2: Why we are using that**To associate mapping data (like an ID to a Name) ensuring highly optimized data retrieval via unique keys rather than numeric indexes 70, 71.
**Step 3: How to use that**Define via {key: value}. Use methods to extract keys, values, or safely get items without causing errors 74, 76.
**Step 4: Give one practical example on that**

```
user = {"name": "Alice", "role": "Admin"}
```

PRAGYAN AI

```
print(user.get("age", "Not Found")) # Safely gets 'age' or defaults
to 'Not Found'
user.update({"age": 30, "city": "NYC"}) # Adds/updates multiple pairs
print(user.keys())   # dict_keys(['name', 'role', 'age', 'city'])
print(user.values()) # dict_values(['Alice', 'Admin', 30, 'NYC'])
print(user.items())  # Returns all key-value tuples
user.pop("city")     # Removes key 'city'
```

## 8. Functions and Arguments

**Step 1: Definition**A function is a modular, reusable block of code defined with the def keyword 77-79. Functions can take inputs (Arguments: Positional, Keyword, Default, and Arbitrary *args/**kwargs) and optionally return values 80-85. A function calling itself is known as Recursion 86, 87.
**Step 2: Why we are using that**To promote code modularity, increase readability, and prevent repetition (Code Reusability). Recursion simplifies complex problems like math factorials 78, 86.
**Step 3: How to use that**Use def func_name(parameters): followed by logic and an optional return statement. Supply default values (arg=value) for Default arguments, pass explicit names (name="Bob") for Keyword arguments, or use *args to capture an unlimited list of arguments 82, 84, 88.
**Step 4: Give one practical example on that**

```
# Function with Default, Arbitrary args, and Return
def process_data(multiplier=2, *numbers):
    result = []
    for num in numbers:
        result.append(num * multiplier)
    return result

# Calling with Positional and Arbitrary arguments
output = process_data(3, 10, 20, 30)
print(output) # [14, 76, 83]

# Recursive Function
def factorial(n):
    if n == 1 or n == 0: return 1
    return n * factorial(n - 1)
```

## 9. Object-Oriented Programming (OOP) Concepts

**Step 1: Definition**OOP is a paradigm designing software around "Objects" (data entities with state) and "Classes" (blueprints) 89, 90. The four core pillars are:

1. **Encapsulation:** Wrapping variables and methods into a single unit (hiding data) 91, 92.
2. **Inheritance:** A child class acquiring properties of a parent class (Single, Multiple, Multilevel, Hierarchical, Hybrid) 93-95.

PRAGYAN AI

3. **Polymorphism:** Using a single entity/method to represent different behaviors depending on context (Method Overriding and Overloading) 93, 96, 97.
4. **Abstraction:** Hiding complex internal details and showing only essential features 91.**Step 2: Why we are using that**To build large, scalable, and maintainable programs. It enhances code security (Encapsulation), aggressive code reuse (Inheritance), and flexibility (Polymorphism) 98, 99.**Step 3: How to use that**Define a class. Use the __init__ constructor to initialize state. Pass self to instance methods. Use super() to trigger parent logic in inheritance 100, 101.**Step 4: Give one practical example on that**

```
# Encapsulation & Class definition
class Vehicle:
    def __init__(self, name):
        self.name = name # Public attribute
        self.__engine_status = "Off" # Encapsulated (Private)
attribute

    def start_engine(self):
        self.__engine_status = "On"
        return f"{self.name} engine is {self.__engine_status}"

# Inheritance & Polymorphism (Overriding)
class Car(Vehicle):
    def __init__(self, name, wheels):
        super().__init__(name) # Inheriting parent constructor
        self.wheels = wheels

    # Polymorphism: Overriding parent behavior if it existed
    def start_engine(self):
        return super().start_engine() + f" with {self.wheels} wheels
ready!"

my_car = Car("Mustang", 4)
print(my_car.start_engine())
```

## 10. Errors and Exception Handling

**Step 1: Definition**Syntax errors occur due to bad formatting (parsing errors), whereas Exceptions (like ZeroDivisionError, ValueError) are runtime disruptions 58, 97, 102, 103. Handling involves intercepting these exceptions using try, except, and finally 95, 104.**Step 2: Why we are using that**If an exception is unhandled, a program will crash abruptly. Exception handling safeguards system resources, ensures graceful failure, and keeps applications running 95, 104, 105.**Step 3: How to use that**Wrap risky code inside try:. Catch specific or general errors using except ExceptionType as e:. Use finally: for cleanup code (e.g., closing files) that strictly executes regardless of whether an error happened 104, 106, 107.**Step 4: Give one practical example on that**

```
try:
```

PRAGYAN AI

```
    x = int(input("Enter number: "))
    result = 10 / x
    print(result)
except ZeroDivisionError as e:
    print("Cannot divide by zero!", e)
except ValueError:
    print("Please enter a valid integer!")
finally:
    print("Execution complete. Cleaning up resources.")
```

## 11. Python Decorators

**Step 1: Definition**Decorators are higher-order functions that modify or extend the behavior of another function without explicitly changing its code 108. They wrap the target function using inner functions 108.**Step 2: Why we are using that**To dynamically add syntactic sugar and reusable functionality (like logging, timing execution, authentication) cleanly and transparently across multiple functions 109, 110.**Step 3: How to use that**Define a function that takes a function as an argument, defines a wrapper function inside it, and returns the wrapper 108. Apply it instantly using the pie syntax @decorator_name directly above the target function 111. Use @functools.wraps to preserve the original function's metadata 112.**Step 4: Give one practical example on that**

```
import functools

# The Decorator
def my_logger(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Executing {func.__name__}...")
        result = func(*args, **kwargs)
        print("Execution finished.")
        return result
    return wrapper

# Applying the decorator
@my_logger
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Architect")
# Output:
# Executing say_hello...
# Hello, Architect!
# Execution finished.
```

PRAGYAN AI

# PRAGYAN AI PYTHON NOTES

## 1. Variables and Data Types

**Step 1: Definition**Variables are reserved memory locations used to store different types of values (like numbers, text, or objects) 3, 4. Python is dynamically typed, meaning you do not explicitly declare the type of a variable before using it 5, 6. Core data types include Integer (whole numbers), Float (decimal numbers), Boolean (True/False), and Strings (text) 7-9.
**Step 2: Why we are using that**Variables are the foundational building blocks of programming, allowing us to store, retrieve, manipulate, and reuse data dynamically throughout a program's lifecycle 3, 4.
**Step 3: How to use that**You create a variable by assigning a value to a name using the assignment operator (=) 6, 10. Variable names must start with a letter or underscore and are case-sensitive 10.
**Step 4: Give one practical example on that**

```
age = 25                        # Integer data type
price = 50.5                    # Float data type
is_active = True                # Boolean data type
message = "Welcome to Python" # String data type
```

## 2. Input and Output Statements

**Step 1: Definition**The input() function prompts the user to enter data via the console and returns it as a string, while the print() function displays output back to the console 11, 12.
**Step 2: Why we are using that**They are essential for building interactive applications where the system must communicate with the user to request information and present calculated results 11, 13.
**Step 3: How to use that**Call input("Prompt") to pause execution and read user input 11. To output data, call print(data). You can format strings elegantly using f-strings (prefixing the string with f and enclosing variables in {}) 12.
**Step 4: Give one practical example on that**

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # Typecasting string to integer
print(f"My name is {name} and I am {age} years old.")
```

## 3. Python Operators

Operators perform operations on values and variables 14, 15. Python contains several types of operators, detailed below.

PRAGYAN AI

### 3.1 Arithmetic Operators

**Step 1: Definition**Arithmetic operators are used to perform mathematical operations: Addition (+), Subtraction (-), Multiplication (*), Division (/), Floor Division (//), Modulo/Remainder (%), and Exponentiation/Power (**) 14, 16.
**Step 2: Why we are using that**To perform basic to advanced mathematical calculations dynamically within our code 16.
**Step 3: How to use that**Place the operator symbol between two numeric operands 16, 17.
**Step 4: Give one practical example on that**

```
a, b = 10, 3
print(a + b)  # 13 (Addition)
print(a / b)  # 3.333 (Division)
print(a // b) # 3 (Floor Division - rounds down)
print(a % b)  # 1 (Modulo - remainder)
print(a ** b) # 1000 (10 to the power of 3)
```

### 3.2 Assignment Operators

**Step 1: Definition**Assignment operators are used to assign values to variables: =, +=, -=, *=, /=, %=, **= 18.
**Step 2: Why we are using that**To initialize or update the value of a variable in a concise manner (e.g., incrementing a counter) 18.
**Step 3: How to use that**Place the variable on the left and the value/expression on the right (e.g., a += 1 is shorthand for a = a + 1) 18.
**Step 4: Give one practical example on that**

```
x = 5
x += 3 # x becomes 8
x *= 2 # x becomes 16
```

### 3.3 Comparison (Relational) Operators

**Step 1: Definition**Comparison operators compare two values and return a boolean result (True or False): Equal (==), Not Equal (!=), Greater Than (>), Less Than (<), Greater/Equal (>=), Less/Equal (<=) 19, 20.
**Step 2: Why we are using that**They evaluate conditions to make decisions in loops and conditional statements 20.
**Step 3: How to use that**Place the operator between two operands. The interpreter evaluates the relationship and returns a boolean 20.
**Step 4: Give one practical example on that**

```
a, b = 5, 10
print(a == b) # False
print(a != b) # True
print(a < b)  # True
```

PRAGYAN AI

### 3.4 Logical Operators

**Step 1: Definition**Logical operators combine conditional statements: and (True if both are true), or (True if at least one is true), and not (reverses the boolean state) 19, 21.
**Step 2: Why we are using that**To check multiple conditions simultaneously for complex decision-making 21.
**Step 3: How to use that**Place and/or between two boolean expressions, or place not before a single expression 21.
**Step 4: Give one practical example on that**

```
x = 5
print(x > 3 and x < 10)  # True (Both conditions met)
print(x > 3 or x > 10)   # True (One condition met)
print(not(x > 3))        # False (Reverses True to False)
```

### 3.5 Bitwise Operators

**Step 1: Definition**Bitwise operators act on operands bit-by-bit: AND (&), OR (|), NOT (~), XOR (^), Left Shift (<<), Right Shift (>>) 19, 22.
**Step 2: Why we are using that**To perform low-level operations directly on binary numbers 19.
**Step 3: How to use that**Apply them to integers; Python evaluates them at the binary (bit) level 19, 22.
**Step 4: Give one practical example on that**

```
a = 10 # Binary: 1010
b = 4  # Binary: 0100
print(a & b) # 0 (Binary AND)
print(a | b) # 14 (Binary OR)
```

### 3.6 Identity Operators

**Step 1: Definition**Identity operators (is, is not) check if two values are located on the exact same part of the computer's memory 22, 23.
**Step 2: Why we are using that**To verify if two variables reference the exact same object, not just if their values are equal 22, 23.
**Step 3: How to use that**Use x is y or x is not y to evaluate memory identity 23.
**Step 4: Give one practical example on that**

```
x1, y1 = 5, 5
print(x1 is y1) # True (Integers map to same memory)
list1, list2 = [24, 25]
print(list1 is list2) # False (Equal values, but separate objects in
memory)
```

### 3.7 Membership Operators

**Step 1: Definition**Membership operators (in, not in) test whether a value or variable is found inside a sequence (like a string, list, tuple, or dictionary) 22, 26.
**Step 2: Why we are using that**To quickly validate if a specific item exists within a collection without writing manual search loops 26.

PRAGYAN AI

**Step 3: How to use that**Use value in sequence to return True if it exists, or False otherwise 27.
**Step 4: Give one practical example on that**

```
message = "Hello World"
print('H' in message) # True
print('z' not in message) # True
```

## 4. Control Flow (Selective Statements)

**Step 1: Definition**Selective statements control the flow of a program based on evaluating boolean conditions. They include if, if-else, if-elif-else, and nested if statements 28, 29.
**Step 2: Why we are using that**To enable decision-making in our software, allowing the program to execute different logic based on varying inputs or states 28, 29.
**Step 3: How to use that**Use if condition: followed by an indented block. Provide fallback logic using elif (else if) for multiple conditions, and an else block for when all conditions fail 30-32.
**Step 4: Give one practical example on that**

```
age = int(input("Enter Age: "))
if age < 18:
    print("Minor")
elif age >= 18 and age < 65:
    print("Adult")
else:
    print("Senior")
```

## 5. Iterative & Jump Statements (Loops)

**Step 1: Definition**Iterative statements repeat a block of code. The for loop iterates over a sequence (lists, strings, ranges), while the while loop reiterates as long as a condition is true 33-37. Jump statements (break, continue, pass) alter loop execution 38, 39.
**Step 2: Why we are using that****Loops to automate** repetitive tasks. Jump statements provide fine-grained control: break completely terminates a loop, continue skips the current iteration, and pass acts as an empty placeholder 38-42.
**Step 3: How to use that**Use for item in sequence: or while condition:. Inside the loop, insert break or continue coupled with if conditions to manipulate the flow 35, 38, 39.
**Step 4: Give one practical example on that**

```
# For loop with Continue
for i in range(1, 6):
    if i == 3:
        continue # Skips printing 3
    print(i) # Prints 1, 2, 4, 5

# While loop with Break
count = 1
while True:
    print(count)
    if count == 3:
```

PRAGYAN AI

```
        break # Terminates loop at 3
    count += 1
```

## 6. String Handling & Methods

**Step 1: Definition**A string is an immutable sequence of Unicode characters 40, 43. Python provides built-in methods to manipulate strings: isalnum(), isalpha(), isdigit(), islower(), isspace(), istitle(), isupper(), replace(), split(), count(), and find() 44, 45.
**Step 2: Why we are using that**To format, validate, clean, and extract insights from textual data easily without building custom logic 44.
**Step 3: How to use that**Access characters via slicing string[start:stop:step] 46, 47. Call methods using dot notation: string.method_name() 45, 48
**Step 4: Give one practical example on that**

```
text = "Python Programming 101"
print(text[0:6])                # Slicing: 'Python'
print(text.isupper())           # False
print(text.replace("101", "Masterclass")) # 'Python Programming
Masterclass'
print(text.split())             # ['Python', 'Programming', '101']
print(text.count('o'))          # 2
```

## 7. Built-in Data Structures

### 7.1 Lists and List Methods

**Step 1: Definition**A list is an ordered, mutable collection that allows duplicate elements 49, 50. Comprehensive methods include: append(), clear(), copy(), count(), extend(), index(), insert(), pop(), remove(), reverse(), and sort() 51, 52.
**Step 2: Why we are using that**To dynamically store and manipulate an ordered sequence of varying data types (e.g., adding or removing items on the fly) 49, 53.
**Step 3: How to use that**Declare lists using []. Call methods via list_name.method() 49, 51. You can also use List Comprehension [expr for item in iterable] for concise creation 54.
**Step 4: Give one practical example on that**

```
data = [25, 55, 56]
data.append(10)         # Adds 10: [25, 55-57]
data.insert(1, 15)      # Inserts at index 1: [2, 25, 55-57]
data.extend([24, 58])    # Appends sequence: [2, 24, 25, 55-58]
data.remove(2)          # Removes value 2
popped = data.pop()     # Removes and returns last item (3)
data.sort()             # Sorts ascending: [2, 24, 55-57]
data.reverse()          # Reverses order: [2, 24, 55-57]
print(data.count(10)) # Returns 1
```

PRAGYAN AI

### 7.2 Tuples and Tuple Methods

**Step 1: Definition**A tuple is an ordered, *immutable* (unchangeable) collection defined using parentheses () 59-61. Key methods are count() and index(), and it supports len() 62, 63.
**Step 2: Why we are using that**To safeguard data that must not change post-creation (e.g., geographic coordinates). They are faster and more memory-efficient than lists 60, 61.
**Step 3: How to use that**Create via (). Read values via indexing. Because they are immutable, packing (grouping items) and unpacking (assigning items to variables) are common operations 64, 65.
**Step 4: Give one practical example on that**

```
dimensions = (1920, 1080, 1080)
print(dimensions.count(1080)) # Returns 2
print(dimensions.index(1920)) # Returns index 0
# Unpacking
w, h, depth = dimensions
```

### 7.3 Sets and Set Methods

**Step 1: Definition**A set is an unordered collection of *unique* elements 66, 67. Operations include add(), remove(), discard(), Union (|), Intersection (&), Difference (-), and Symmetric Difference (^) 66, 68, 69.
**Step 2: Why we are using that**To perform mathematical set operations rapidly and eliminate duplicate values from other collections 68.
**Step 3: How to use that**Define using {} or set(). Apply mathematical operations using their respective methods or operator symbols 66, 68.
**Step 4: Give one practical example on that**

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set1.add(6)             # {1, 2, 3, 6}
set1.discard(6)         # Removes 6 safely
print(set1 | set2)      # Union: {1, 2, 3, 4, 5}
print(set1 & set2)      # Intersection: {3}
print(set1 - set2)      # Difference: {1, 2}
```

### 7.4 Dictionaries and Dictionary Methods

**Step 1: Definition**A dictionary is an unordered, mutable collection of key-value pairs 70, 71. Methods include: clear(), copy(), fromkeys(), get(), items(), keys(), pop(), popitem(), setdefault(), update(), and values() 72-75.
**Step 2: Why we are using that**To associate mapping data (like an ID to a Name) ensuring highly optimized data retrieval via unique keys rather than numeric indexes 70, 71.
**Step 3: How to use that**Define via {key: value}. Use methods to extract keys, values, or safely get items without causing errors 74, 76.
**Step 4: Give one practical example on that**

```
user = {"name": "Alice", "role": "Admin"}
```

PRAGYAN AI

```
print(user.get("age", "Not Found")) # Safely gets 'age' or defaults
to 'Not Found'
user.update({"age": 30, "city": "NYC"}) # Adds/updates multiple pairs
print(user.keys())   # dict_keys(['name', 'role', 'age', 'city'])
print(user.values()) # dict_values(['Alice', 'Admin', 30, 'NYC'])
print(user.items())  # Returns all key-value tuples
user.pop("city")     # Removes key 'city'
```

## 8. Functions and Arguments

**Step 1: Definition**A function is a modular, reusable block of code defined with the def keyword 77-79. Functions can take inputs (Arguments: Positional, Keyword, Default, and Arbitrary *args/**kwargs) and optionally return values 80-85. A function calling itself is known as Recursion 86, 87.
**Step 2: Why we are using that**To promote code modularity, increase readability, and prevent repetition (Code Reusability). Recursion simplifies complex problems like math factorials 78, 86.
**Step 3: How to use that**Use def func_name(parameters): followed by logic and an optional return statement. Supply default values (arg=value) for Default arguments, pass explicit names (name="Bob") for Keyword arguments, or use *args to capture an unlimited list of arguments 82, 84, 88.
**Step 4: Give one practical example on that**

```
# Function with Default, Arbitrary args, and Return
def process_data(multiplier=2, *numbers):
    result = []
    for num in numbers:
        result.append(num * multiplier)
    return result

# Calling with Positional and Arbitrary arguments
output = process_data(3, 10, 20, 30)
print(output) # [14, 76, 83]

# Recursive Function
def factorial(n):
    if n == 1 or n == 0: return 1
    return n * factorial(n - 1)
```

## 9. Object-Oriented Programming (OOP) Concepts

**Step 1: Definition**OOP is a paradigm designing software around "Objects" (data entities with state) and "Classes" (blueprints) 89, 90. The four core pillars are:

1. **Encapsulation:** Wrapping variables and methods into a single unit (hiding data) 91, 92.
2. **Inheritance:** A child class acquiring properties of a parent class (Single, Multiple, Multilevel, Hierarchical, Hybrid) 93-95.

PRAGYAN AI

3. **Polymorphism:** Using a single entity/method to represent different behaviors depending on context (Method Overriding and Overloading) 93, 96, 97.
4. **Abstraction:** Hiding complex internal details and showing only essential features 91.**Step 2: Why we are using that**To build large, scalable, and maintainable programs. It enhances code security (Encapsulation), aggressive code reuse (Inheritance), and flexibility (Polymorphism) 98, 99.**Step 3: How to use that**Define a class. Use the __init__ constructor to initialize state. Pass self to instance methods. Use super() to trigger parent logic in inheritance 100, 101.**Step 4: Give one practical example on that**

```
# Encapsulation & Class definition
class Vehicle:
    def __init__(self, name):
        self.name = name # Public attribute
        self.__engine_status = "Off" # Encapsulated (Private)
attribute

    def start_engine(self):
        self.__engine_status = "On"
        return f"{self.name} engine is {self.__engine_status}"

# Inheritance & Polymorphism (Overriding)
class Car(Vehicle):
    def __init__(self, name, wheels):
        super().__init__(name) # Inheriting parent constructor
        self.wheels = wheels

    # Polymorphism: Overriding parent behavior if it existed
    def start_engine(self):
        return super().start_engine() + f" with {self.wheels} wheels
ready!"

my_car = Car("Mustang", 4)
print(my_car.start_engine())
```

## 10. Errors and Exception Handling

**Step 1: Definition**Syntax errors occur due to bad formatting (parsing errors), whereas Exceptions (like ZeroDivisionError, ValueError) are runtime disruptions 58, 97, 102, 103. Handling involves intercepting these exceptions using try, except, and finally 95, 104.**Step 2: Why we are using that**If an exception is unhandled, a program will crash abruptly. Exception handling safeguards system resources, ensures graceful failure, and keeps applications running 95, 104, 105.**Step 3: How to use that**Wrap risky code inside try:. Catch specific or general errors using except ExceptionType as e:. Use finally: for cleanup code (e.g., closing files) that strictly executes regardless of whether an error happened 104, 106, 107.**Step 4: Give one practical example on that**

```
try:
```

PRAGYAN AI

```
    x = int(input("Enter number: "))
    result = 10 / x
    print(result)
except ZeroDivisionError as e:
    print("Cannot divide by zero!", e)
except ValueError:
    print("Please enter a valid integer!")
finally:
    print("Execution complete. Cleaning up resources.")
```

## 11. Python Decorators

**Step 1: Definition**Decorators are higher-order functions that modify or extend the behavior of another function without explicitly changing its code 108. They wrap the target function using inner functions 108.**Step 2: Why we are using that**To dynamically add syntactic sugar and reusable functionality (like logging, timing execution, authentication) cleanly and transparently across multiple functions 109, 110.**Step 3: How to use that**Define a function that takes a function as an argument, defines a wrapper function inside it, and returns the wrapper 108. Apply it instantly using the pie syntax @decorator_name directly above the target function 111. Use @functools.wraps to preserve the original function's metadata 112.**Step 4: Give one practical example on that**

```
import functools

# The Decorator
def my_logger(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Executing {func.__name__}...")
        result = func(*args, **kwargs)
        print("Execution finished.")
        return result
    return wrapper

# Applying the decorator
@my_logger
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Architect")
# Output:
# Executing say_hello...
# Hello, Architect!
# Execution finished.
```

PRAGYAN AI